

# 紙芝居アプリ内部仕様書

## 目次

|                                   |    |
|-----------------------------------|----|
| この文書で使う用語 .....                   | 3  |
| Scratch/TurboWarpのproject構造 ..... | 3  |
| cloneとActor .....                 | 4  |
| 紙芝居DSLと実行 .....                   | 4  |
| 変数とblock .....                    | 5  |
| 1. 文書の範囲と実装基準 .....               | 5  |
| 1.1 実装スナップショット .....              | 6  |
| 1.2 使用する機能拡張 .....                | 7  |
| 2. 成果物プロファイル .....                | 7  |
| 3. SB3・台本変換ビルダー .....             | 8  |
| 3.1 導入 .....                      | 8  |
| 3.2 CLI .....                     | 8  |
| 3.3 JavaScript API .....          | 9  |
| 3.4 アセットマニフェスト .....              | 10 |
| 3.5 安全性と再現性 .....                 | 11 |
| 4. SB3の構成 .....                   | 11 |
| 4.1 target一覧 .....                | 12 |
| 4.2 アクターへ命令を届けるしくみ .....          | 12 |
| 4.3 target間の責務 .....              | 16 |
| 5. 変数とlist .....                  | 16 |
| 5.1 Scratch変数 .....               | 16 |
| 5.2 Scratch list .....            | 16 |
| 5.3 runtime variable .....        | 17 |
| 5.4 thread variable .....         | 18 |
| 6. event、カスタムブロック、呼出し関係 .....     | 19 |
| 6.1 event hat一覧 .....             | 19 |
| 6.2 カスタムブロック定義一覧 .....            | 22 |
| 6.3 主要な呼出し経路 .....                | 26 |

|                         |    |
|-------------------------|----|
| 7. broadcastと状態遷移 ..... | 27 |
| 7.1 message一覧 .....     | 27 |
| 7.2 状態遷移 .....          | 28 |
| 8. 検証 .....             | 30 |
| 8.1 変更対象ごとのテスト .....    | 30 |
| 8.2 標準チェック .....        | 30 |
| 9. 公開 .....             | 31 |
| 9.1 GitHub Pages .....  | 31 |
| 9.2 npmパッケージ .....      | 31 |
| 10. トラブルシューティング .....   | 32 |
| 11. ライセンスと秘密情報 .....    | 33 |
| 12. 関連ドキュメント .....      | 33 |

# 紙芝居アプリ内部仕様書

Copyright © 2026 Hiroya Kubo. この文書は [CC BY-SA 4.0](#) で提供します。

この文書は、TMPose紙芝居の汎用アプリSB3、成果物、SB3・台本変換ビルダー、検証、公開の内部仕様を、現在の実装に対応させて記録します。アプリを変更する手順は [ソフトウェア開発者向け資料](#)、台本の外部仕様は [台本DSLマニュアル](#)と [コマンドリファレンス](#)を参照してください。

対象アプリ/DSL: `kamishibai=3.1`

過去のバージョンからの変更は [history.md](#) を参照してください。

## この文書で使う用語

本書ではScratch/TurboWarpの用語と、このアプリ固有の用語を次の意味で使います。表中の等幅書体（`Stage`、`script`、`action=` など）は、target名、変数名、DSL記法として実装に現れる正確な綴りを表します。通常書体の用語は、概念または分類名を表します。詳細なデータ構造や処理は、表に示した後続の章で説明します。

## Scratch/TurboWarpのproject構造

| 用語                 | この文書での意味                                                                                     |
|--------------------|----------------------------------------------------------------------------------------------|
| project            | Stage、sprite、block、変数、画像、音声などをまとめたScratch/TurboWarp作品全体                                      |
| SB3                | projectを1ファイルに格納するScratch 3形式。このリポジトリではbuildによって生成する成果物                                      |
| target             | project内でblock、変数、costumeなどを所有する実行単位。1つのStage targetと、0個以上のsprite targetがある                  |
| <code>Stage</code> | projectに1つだけある舞台のtarget。このアプリでは背景表示に加え、台本解析と実行状態の統括を担う。詳細は「 <a href="#">SB3の構成</a> 」（第4章）を参照 |
| sprite             | 舞台上に表示・移動でき、costume、sound、block、変数を持てるtarget                                                 |

## clone と Actor

| 用語                                     | この文書での意味                                                                                         |
|----------------------------------------|--------------------------------------------------------------------------------------------------|
| clone                                  | 実行中に sprite から作る複製。同じ block を共有する一方、スプライトローカル変数には clone ごとの値を持てる                                 |
| <code>Actor</code> target / アクター スプライト | アクターを clone として生成するための sprite 雛形。この project では target 名が <code>Actor</code>                      |
| アクター                                   | <code>Actor</code> target から作られ、 <code>actorName</code> で区別される個々の clone。物語上の登場人物として action を実行する |

## 紙芝居 DSL と実行

| 用語                  | この文書での意味                                                                                                                                           |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| asset               | Asset Manager へ名前付きで登録する画像、音声、text。SB3 内の costume などと外部 URL の resource を同じ方法で参照できる                                                                 |
| 紙芝居 DSL / 台本ファイル    | asset、アクター、scene、actionなどをテキストで定義する言語と、その言語で記述したファイル                                                                                               |
| <code>script</code> | 読み込んだ台本を保持する runtime variable。外部ファイルと組み込み台本は、ここから同じ処理経路へ入る                                                                                         |
| scene               | 台本を --- で区切った実行単位。内部では最初の区切りより前を scene 0 として扱う                                                                                                     |
| command             | 台本の <code>key=value</code> 形式の 1 行。 <code>exec command %s %s</code> が key に応じて設定、定義、action 登録などを行う                                                 |
| action              | scene 内で順番に実行する演出命令。Stage が実行する Stage action と、アクターへ送る Actor action がある                                                                            |
| action envelope     | Actor action の宛先、command、引数を入れる <code>actionTarget</code> 、 <code>actionCommand</code> 、 <code>actionParam</code> 、 <code>actionParam2</code> のまとめ |
| message / broadcast | Scratch / TurboWarp の event 配送機構。broadcast すると、その message に対応する hat block がある target や clone で処理が始まる                                               |



## 変数と block

| 用語                      | この文書での意味                                                                                                                                         |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Scratch 変数<br>/list     | SB3 に保存され、Stage または sprite が所有する値と配列                                                                                                             |
| runtime<br>variable     | Runtime Variables 機能拡張が保持し、project 全体から参照する共有値                                                                                                   |
| thread<br>variable      | Thread Variables 機能拡張が、カスタムブロック呼出し単位で保持する一時値                                                                                                     |
| event hat/<br>hat block | green flag、broadcast 受信、click などの event を受けて処理を開始する、stack 最上部の block                                                                             |
| カスタムブロッ<br>ク            | project 内で定義し、名前と引数によって呼び出す処理。一般的なプログラミング言語の procedure に相当する                                                                                     |
| block ID                | SB3 の <code>targets[].blocks</code> 内で block を識別する key。実装スナップショット内の調査にだけ使い、版をまたぐ永続IDとしては扱わない。詳細は「 <a href="#">event、カスタムブロック、呼出し関係</a> 」（第6章）を参照 |

## 1. 文書の範囲と実装基準

アプリ内部構造の正本は `app/project.source.json` です。本書の target、変数、list、broadcast、hat、カスタムブロック定義は、このファイルから抽出した現在の構造を 記載しています。配布用 `kamishibai.sb3` は `app/` から生成する成果物であり、本書の 調査元にはしません。

## 1.1 実装スナップショット

| 項目                     | 件数   |
|------------------------|------|
| target (Stageを含む)      | 8    |
| block                  | 1444 |
| event hat              | 39   |
| カスタムブロック定義             | 40   |
| Scratch変数              | 6    |
| Scratch list           | 11   |
| broadcast message      | 18   |
| 静的な runtime variable 名 | 16   |
| 静的な thread variable 名  | 36   |
| TurboWarp 機能拡張         | 12   |

本書に掲載する block ID の意味と安定性は 「event、カスタムブロック、呼出し関係」(第6章) で説明します。

## 1.2 使用する機能拡張

| ID                                       | 役割                                          | 取得形態    |
|------------------------------------------|---------------------------------------------|---------|
| <code>sipconsole</code>                  | デバッグ用 console                               | Gallery |
| <code>lmsTempVars2</code>                | runtime variable / thread variable          | Gallery |
| <code>strings</code>                     | 文字列処理                                       | Gallery |
| <code>kubohiroyaassetmanager</code>      | 画像・音声の登録と Loading 進捗                        | 埋め込み    |
| <code>tmpose</code>                      | カメラ姿勢認識                                     | 埋め込み    |
| <code>localStorage</code>                | 台本のローカル保存                                   | Gallery |
| <code>kubohiroyatextlines</code>         | 行単位の台本処理                                    | 埋め込み    |
| <code>kubohiroyaruntimeexpression</code> | 分岐条件式の評価                                    | 埋め込み    |
| <code>kubohiroyaasyncinput</code>        | key / touch 入力と scene 移動                    | 埋め込み    |
| <code>lmsTimers</code>                   | <code>wait</code> と時間ベース actor action のタイマー | Gallery |
| <code>files</code>                       | 外部台本ファイルの選択                                 | Gallery |
| <code>text</code>                        | テキスト描画・アニメーション                              | Gallery |

埋め込み拡張の由来、固定 commit、SHA-256 は `app/embedded-extensions.json` を正本とし、更新方法は [sb3-toolchain](#) のワークフロー に従います。

## 2. 成果物プロファイル

紙芝居の成果物は、台本と物語固有アセットをどこに保持するかで分けます。

| プロファイル               | 例                           | 台本    | 物語固有アセット | 主な用途                 |
|----------------------|-----------------------------|-------|----------|----------------------|
| <code>generic</code> | <code>kamishibai.sb3</code> | 非埋め込み | 非埋め込み    | 本体が配布する汎用雛形          |
| <code>editor</code>  | <code>_urashima.sb3</code>  | 非埋め込み | 埋め込み     | 物語作成者の編集・動作確認        |
| <code>player</code>  | <code>urashima.sb3</code>   | 埋め込み  | 埋め込み     | 配布・再生、Packager Web 版 |

`generic` は `app/` から生成し、特定の物語を含めません。builder API と CLI が受け付ける `profile` は `editor` または `player` です。

`editor` と `player` は同じベース SB3、台本、アセットロックから生成します。両者の 変換済み台本とアセット参照を分岐させません。`player` は組み込み台本を予約変数へ保存し、タイトル操作後にファイル選択なしで開始します。

`player` へ台本とアセットを組み込んでも、TMPose モデル、カメラ、外部サービスまで自動的にオフライン化されるわけではありません。残るオンライン依存は成果物 manifest と 公開ページへ明記します。

## 3. SB3・台本変換ビルダー

### 3.1 導入

利用可能なバージョンを確認し、消費側で明示的に固定します。

```
npm view @kubohiroya/tmpose-kamishibai version
pnpm add --save-exact @kubohiroya/tmpose-kamishibai@<VERSION>
```

生成した lockfile を commit し、CI では `pnpm install --frozen-lockfile` を使います。

### 3.2 CLI

```
pnpm exec tmpose-kamishibai build-sb3 \
  --base kamishibai.sb3 \
  --script source.txt \
  --assets assets.lock.json \
  --output dist/sample \
  --profile editor
```

`--output` は拡張子を含まないベース名です。次の 3 ファイルを同じ transaction として 生成します。

```
dist/sample.sb3
dist/sample.txt
dist/sample.manifest.json
```

| オプション                              | 意味                                   |
|------------------------------------|--------------------------------------|
| <code>--allow-file-root DIR</code> | <code>file:</code> の許可ルートを追加。複数回指定可能 |
| <code>--allow-http</code>          | 平文HTTPを明示的に許可                        |
| <code>--timeout-ms N</code>        | 1 リクエストのタイムアウト                       |
| <code>--max-asset-bytes N</code>   | 1 アセットの最大バイト数                        |
| <code>--max-script-bytes N</code>  | 組み込み台本の最大バイト数                        |
| <code>--max-redirects N</code>     | HTTPリダイレクト上限                         |

完全な一覧は `pnpm exec tmpose-kamishibai --help` で確認します。

### 3.3 JavaScript API

```
import {
  Sb3BuilderError,
  buildSb3Bundle,
  validateAssetManifest,
  validateBundle,
} from '@kubohiroya/tmpose-kamishibai/builder';

const result = await buildSb3Bundle({
  baseSb3: 'kamishibai.sb3',
  sourceScript: 'source.txt',
  assetManifest: 'assets.lock.json',
  outputDirectory: 'dist',
  outputName: 'sample',
  profile: 'editor',
});

console.log(result.outputPaths);
```

`baseSb3` と `sourceScript` にはファイルパスまたは `file:` URL を指定できます。`assetManifest` にはファイルパス、`file:` URL、または検証対象の JavaScript オブジェクトを指定できます。相対 `file:` を含むオブジェクトでは `manifestBaseDirectory` も指定します。

ネットワーク・ファイル取得は `allowedFileRoots`、`allowHttp`、`requestTimeoutMs`、`maxAssetBytes`、`maxRedirects` で制限できます。`player` の組み込み台本上限は `maxEmbeddedScriptBytes` で変更できます。

`buildSb3Bundle` は `manifest` と `outputPaths` を返します。入力・アセット・出力の問題は `Sb3BuilderError` として処理段階とアセット情報を保持します。

### 3.4 アセットマニフェスト

入力 manifest は `formatVersion: 1` と 1 件以上の `assets` を持ちます。

```
{
  "formatVersion": 1,
  "assets": [
    {
      "name": "forest",
      "uri": "file:assets/forest.svg",
      "kind": "backdrop",
      "target": "@stage",
      "sb3Name": "森",
      "contentType": "image/svg+xml",
      "dataFormat": "svg",
      "size": 1234,
      "sha256": "<64文字の16進数>",
      "license": "CC-BY-4.0: https://creativecommons.org/licenses/by/4.0/",
      "metadata": {
        "bitmapResolution": 1,
        "rotationCenterX": 240,
        "rotationCenterY": 180
      }
    }
  ]
}
```

| kind        | target | 変換後の台本参照                   |
|-------------|--------|----------------------------|
| backdrop    | @stage | backdrop:<sb3Name>         |
| costume     | スプライト名 | costume:<target>:<sb3Name> |
| stageSound  | @stage | sound:@stage:<sb3Name>     |
| spriteSound | スプライト名 | sound:<target>:<sb3Name>   |

DSL 名、同一 target 内の SB3 名、既存 SB3 のアセット名は重複できません。`license` には素材の ライセン  
スまたは利用条件の識別情報と参照先を記録します。

### 3.5 安全性と再現性

- `file:` は既定でmanifestのディレクトリ以下だけを許可し、`..` やsymlinkによる脱出を拒否する
- HTTPSを既定とし、平文HTTPは明示的に許可した場合だけ取得する
- Content-Type、実サイズ、ロック済みサイズ、SHA-256、timeout、redirectを検証する
- ZIP entry 順、timestamp、圧縮設定、JSON表現を固定する
- SB3、変換済み台本、出力manifestの対応を確定前に再検証する
- 3成果物を一時領域で生成し、すべて成功した場合だけ置換する
- 失敗時は既存成果物を保持または復元する

同じ入力、固定依存、設定から生成したSB3、台本、manifestはbit-for-bitで一致しなければなりません。

## 4. SB3の構成

---

Scratch/TurboWarpのprojectは、1つの**Stage (ステージ)** targetと、0個以上のsprite targetから構成されます。Stage targetはproject全体の舞台を表し、背景 (backdrop) を表示します。Stage自身にblock、variable、listを持てますが、spriteのように座標を変えて動かしたり、cloneを作ったりする対象ではありません。

このアプリでは、Stage targetを舞台の表示だけでなく、紙芝居全体の制御役として使います。Stageに置かれたblock群が、台本の読込・解析、assetとactorの生成、sceneとactionの実行、カメラと入力、画面遷移、共有runtime状態を統括します。したがって、本書やシーケンス図で `Stage` と書いた場合は、画面に見える舞台だけでなく、このStage targetとそこに置かれた 制御用block群を指します。

## 4.1 target一覧

| target              | 種別        | 役割                                             | 初期costume/<br>sound   |
|---------------------|-----------|------------------------------------------------|-----------------------|
| Stage               | Stage     | 初期化、台本解析、scene/action実行、カメラ、入力、遷移を統括           | Title, Stars          |
| Actor               | sprite 雛形 | 物語上の登場人物ごとに clone され、移動・見た目・音・時間 action を実行    | button1 / pop         |
| prompt              | UI sprite | 操作案内、pose案内、台本エラーを Asset Manager の costume で表示 | ui-placeholder        |
| openButton          | UI sprite | 外部台本ファイルを選択して startStory へ渡す                   | ui-placeholder        |
| reloadButton        | UI sprite | 保存済みの直前の台本を再読込する                               | ui-placeholder        |
| showTitleButton     | UI sprite | menu から title へ戻す                              | ui-placeholder        |
| Loading             | UI sprite | Asset Manager の読込開始・進捗・完了に合わせて costume を表示     | loading / Chirp       |
| LoadingBubbleAnchor | UI sprite | Loading 進捗メッセージ用の speech bubble 位置を固定          | loading-bubble-anchor |

Actor の本体は非表示で、clone だけを登場人物として表示します。prompt、3つの menu button、Loading、LoadingBubbleAnchor の実画像は、台本の ui.\* 設定または組み込み fallback から Asset Manager へ登録します。

## 4.2 アクターへ命令を届けるしくみ

Scratch/TurboWarp で1つの sprite から複数の登場人物を作る場合、clone ごとに スプライトローカル変数の識別子を持たせれば、同じ block を共有しながら個体を区別できます。このアプリはその考え方を、clone へ命令を届けるところまで拡張しています。

本書では、clone の雛形として使う Actor target を**アクターズプライト**、そこから作られ、スプライトローカル変数の actorName を割り当てられた各 clone を**アクター**と呼びます。アクターズプライトの本体は表示せず、物語の登場人物として表示するのはアクターだけです。すべてのアクターは同じ block 定義を共有し、actorName によって互いを区別します。

Stage からアクターへ命令を届ける流れは次のとおりです。

1. Stage が DSL の Actor action を、宛先となるアクター名、command、引数に分解する



2. `actionTarget`、`actionCommand`、`actionParam`、`actionParam2` という runtime variable へ それぞれを格納する。このまとまりを本書では **action envelope** と呼ぶ
3. Stage が `execActorAction` を broadcast する
4. すべての `Actor` clone が message を受け取り、自分の `actorName` が `actionTarget` の 対象に含まれるかを調べる
5. 対象になったアクターだけが、`actionCommand` を入れ子の条件分岐で振り分け、移動、見た目、音、時間に関する処理を実行する

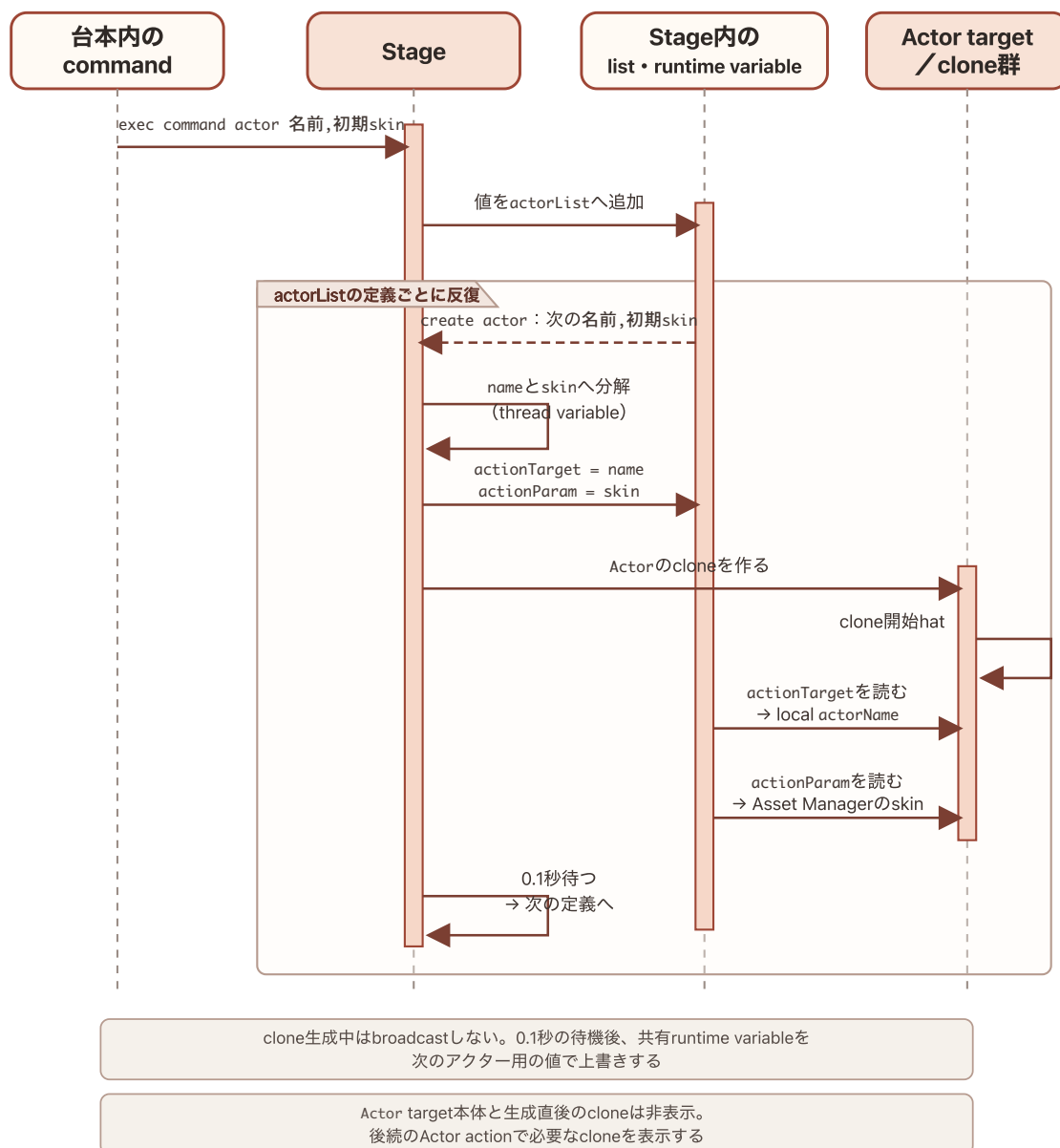
つまり、action envelope が「誰に・何を・どの引数で実行させるか」を表し、broadcast が その命令を全アクターへ配送します。ここでいう command は、アクターに対する メソッドに相当しますが、TurboWarp 上ではカスタムブロックと条件分岐で実装されています。

通常の Scratch project では costume や sound は sprite に属します。このアプリでは TurboWarp Asset Manager を使い、SB3 内の costume、backdrop、sound、text や、外部 URL の画像・音声を、名前を持つ asset として 登録します。アクターの見た目や音の action はこの asset 名を参照するため、振る舞いを実装する アクタースプライトと、実際に使う画像・音声を分けて組み合わせられます。

台本 DSL は、この asset の定義、アクターの定義、scene ごとにアクターへ送る action の定義を テキストで記述するための言語です。TurboWarp で作られたこのアプリは DSL を解析し、asset を 読み込み、アクターを生成し、action envelope と broadcast を使って命令を実行します。したがって、このアプリ全体は、紙芝居 DSL を解析・実行する処理系であり、その実行基盤（runtime）でもあります。

## 台本からアクター clone 生成までのシーケンス

アクターは、台本準備時に `actor=` command から生成します。これは生成済みのアクターへ Actor action を送る処理とは実行時期もデータの渡し方も異なるため、別の図に示します。



### 台本の actor 定義からアクター clone を生成するシーケンス

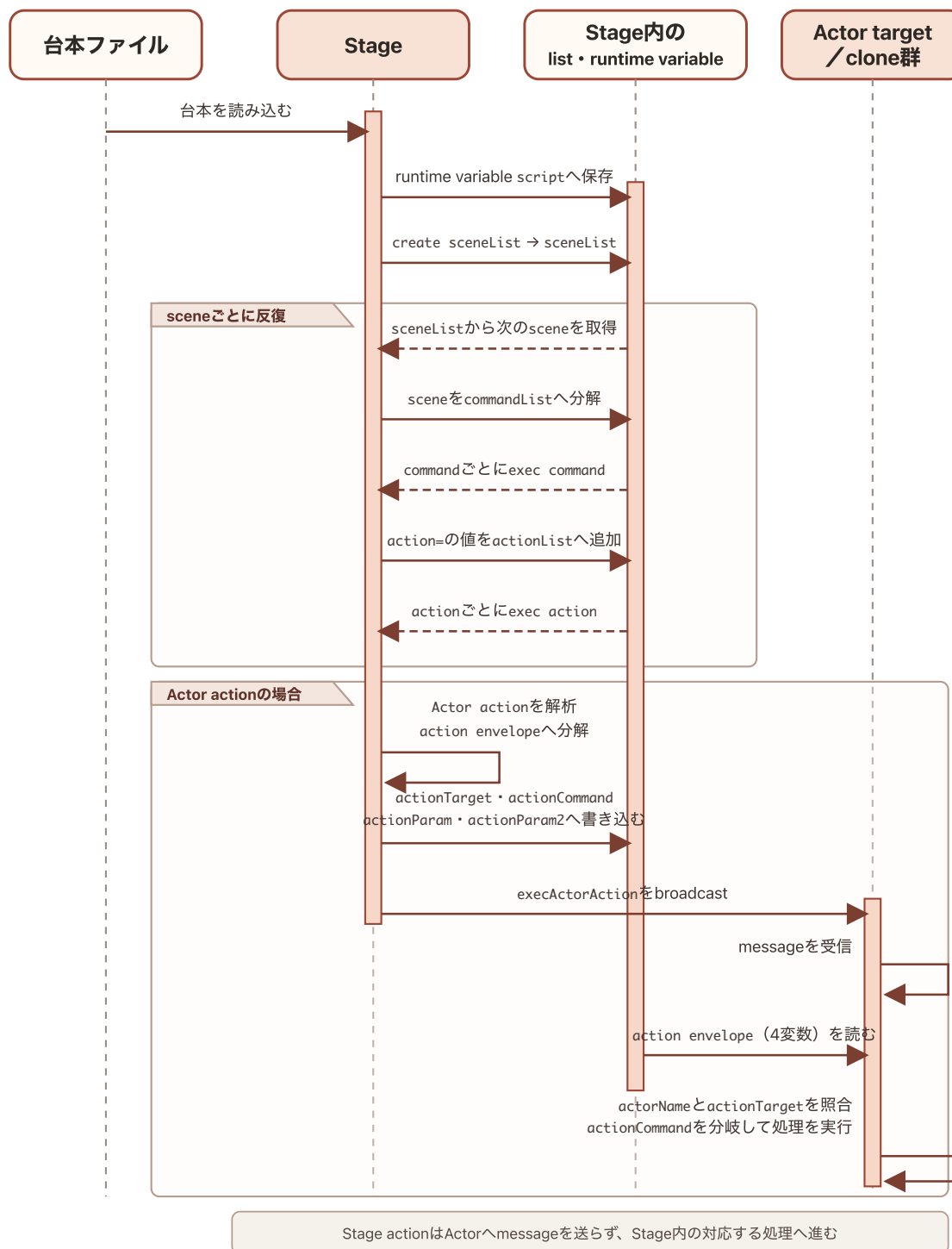
`exec command %s %s` は `actor=` の値を `actorList` へ追加します。各値は `アクター名,初期skin名` の形式です。その後、Stage の `create actor` が `actorList` を順に読み、各値を thread variable の `name` と `skin` へ分けます。Stage は共有 runtime variable の `actionTarget` へ `name`、`actionParam` へ `skin` を設定してから、Actor target の clone を 作ります。

clone 開始 hat は、`actionTarget` をその clone のスプライトローカル変数 `actorName` へ保存し、`actionParam` を TurboWarp Asset Manager へ渡して初期 skin を設定します。Stage は clone を 作るたびに 0.1 秒待ち、この初期化が走る機会を設けてから、共有 runtime variable を次の アクター用の値で上書きします。clone 生成時には `execActorAction` を broadcast しません。

Actor target 本体は clone の雛形として非表示です。clone はこの表示状態を引き継ぐため、生成直後も非表示であり、scene の後続の Actor action によって必要な時点で表示されます。

## 台本から Actor action までのシーケンス

台本ファイルから Actor 内の処理までを、データの変換と実行の順に並べると次のようになります。 外部ファイルだけでなく、再生用SB3へ組み込んだ台本も `script` へ入った後は同じ経路を通ります。



### 台本ファイルからアクターでの命令実行までのシーケンス

`create sceneList` は `script` を scene 単位に分けます。 `exec scene # %s with %s` は現在の scene を 行単位の `commandList` に分け、 `exec command %s %s` で `command` を順に処理します。このうち `action=` の値を `actionList` へ集め、 `exec actionList` が各 `action` を `exec action %s` へ渡します。 Stage action は Stage 内で実行されま

す。Actor actionでは、Stageが `actionTarget`、`actionCommand`、`actionParam`、`actionParam2` へ `action envelope` を書き込んでから、`execActorAction` を broadcast します。messageを受信した各 Actor cloneは4変数を読み、自分の `actorName` と宛先を照合します。実際にcommandを分岐して処理するのは、宛先に一致したアクターだけです。

### 4.3 target間の責務

Stageは台本を `sceneList`、`commandList`、`actionList` へ段階的に変換します。さらにsceneと共有runtime状態を管理し、Stage actionを実行します。Actor cloneの責務は、生成時に自分の `actorName` と初期skinを確定し、その後、「アクターへ命令を届けるしくみ」(4.2節) で配送されたActor actionのうち自分を対象とするものを実行することです。

UI spriteは表示状態を `showMenu` / `hideMenu`、`showPrompt` / `hidePrompt`、Asset ManagerのLoading messageで受け取ります。台本の実行状態をUI sprite側へ複製せず、Stageを状態の所有者とします。

## 5. 変数とlist

変数は、SB3へ永続化されるScratch変数/list、threadごとの一時値、project全体で共有するruntime variableの3種類に分けます。

### 5.1 Scratch 変数

| 所有者   | 変数                                    | 初期値                     | 役割                                     |
|-------|---------------------------------------|-------------------------|----------------------------------------|
| Stage | ポーズ認識                                 | 0                       | pose認識中の表示・互換用状態                       |
| Stage | チャージ                                  | 0                       | pose成立までのcharge表示・互換用状態                |
| Stage | <code>actionIndex</code>              | 1                       | 現在処理する <code>actionList</code> の位置     |
| Stage | <code>poseIndex</code>                | 1                       | 現在処理する <code>poseList</code> の位置       |
| Stage | <code>__tmpose_embedded_script</code> | 空文字                     | <code>player</code> profileの組み込み台本予約領域 |
| Actor | <code>actorName</code>                | <code>_template_</code> | cloneが担当する台本上のactor名                   |

`__tmpose_embedded_script` はStageに一つだけ存在し、monitorを持ちません。`generic` と `editor` では空、`player` ではbuilderが変換済み台本を設定します。

### 5.2 Scratch list

すべてStage所有で、汎用SB3では空の初期状態です。

| list           | 役割                         |
|----------------|----------------------------|
| skinList       | actor action 中の costume 候補 |
| poseList       | pose action 中の認識 label 候補  |
| soundList      | actor action 中の sound 候補   |
| actionList     | 現在 scene の action 列        |
| sceneList      | 台本から抽出した scene block       |
| commandList    | scene の前に評価する設定 command    |
| actorList      | 台本から生成する actor 定義          |
| assetList      | 登録する asset 定義              |
| durationList   | pose 候補ごとの継続時間             |
| sceneLabelList | scene label と index の対応    |
| lines          | Text Lines で分割した台本行        |

## 5.3 runtime variable

静的な名前を持つ runtime variable は次の 16 個です。

| 変数                                                                                                               | 生存期間／役割                                                                                           |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>script</code>                                                                                              | 読み込んだ変換済み台本。title、reload、 <code>startStory</code> 間で共有                                            |
| <code>version</code>                                                                                             | 台本の <code>kamishibai</code> version                                                               |
| <code>startSceneIndex</code>                                                                                     | 台本で指定した開始 scene                                                                                   |
| <code>sceneIndex</code>                                                                                          | 現在実行中の scene index                                                                                |
| <code>actionTarget</code> , <code>actionCommand</code> , <code>actionParam</code> ,<br><code>actionParam2</code> | Stage から Actor clone へ渡す action envelope                                                          |
| <code>nextSceneLabel</code>                                                                                      | key/touch 入力が必要した遷移先 scene label                                                                  |
| <code>skipMode</code>                                                                                            | <code>Space</code> 、 <code>Right</code> 、 <code>Down</code> による未消費の進行要求                           |
| <code>skipContext</code>                                                                                         | <code>title</code> 、 <code>action</code> 、 <code>pose</code> 、 <code>scene</code> のどの境界が必要を消費できるか |
| <code>poseRecog</code> , <code>poseCharge</code> , <code>poseIdle</code>                                         | pose 認識のしきい値、charge 時間、idle 時間                                                                    |
| <code>loadingCostume</code>                                                                                      | Loading sprite へ適用する costume 名                                                                    |
| <code>message</code>                                                                                             | Loading bubble へ表示する現在の進捗文言                                                                       |

このほか、`exec command %s %s` は DSL で指定された runtime variable 名を動的に設定します。分岐条件は `branch:<branchName>` という名前で保存します。この 2 系列は入力から名前が決まるため、静的な 16 個には数えません。

## 5.4 thread variable

カスタムブロック呼出しごとに分離され、呼出し終了後に共有状態として残さない値です。

| 用途                | 名前                                                                           |
|-------------------|------------------------------------------------------------------------------|
| 共通 loop・文字列処理     | index, length, line, lineIndex, name, value, key, keyValue                   |
| 台本解析              | sceneBlock, sceneLabel, sceneLabelList, condition, conditionList             |
| asset/actor 生成    | asset, assetList, resourceId, actor, actorName, skin                         |
| action 実行         | action, actionResult, actionListResult, stageActionResult, actorActionResult |
| command/branch/入力 | commandIndex, branchIndex, keyId, hasRead                                    |
| cover             | cover, coverBackground, coverBgm                                             |
| Actor clone       | x, y, scale                                                                  |
| その他               | durationList, sceneIndex                                                     |

runtime variable と同名の `sceneIndex` thread variable は、カスタムブロック内の局所的な 引数・計算値です。project 全体の現在 scene は runtime variable 側だけを正本とします。

## 6. event、カスタムブロック、呼出し関係

表の `target` は block を所有する Stage または sprite、`ID` はその `target` の `blocks` object にある key です。SB3 内では `project.json` の `targets[].blocks`、このリポジトリ では `app/project.source.json` の同じ位置に保存されます。ID は opcode や表示名ではなく、この実装スナップショット内の block を特定するための内部識別子です。

`sb3-toolchain` の build と import は block ID を新規採番せず、入力に含まれる ID を保持します。既存 block を再生成しない TurboWarp 上の編集・保存でも通常は保持されます。一方、block の 削除と再作成、複製や copy & paste による新しい block の作成、`target/project` の import など block が再生成されると ID は変わります。したがって、ID は外部仕様、永続 ID、他の版をまたぐ 参照には使いません。アプリを編集して ID が変わった場合は、本章も現在の `app/project.source.json` に合わせて更新します。

### 6.1 event hat 一覧

`procedures_definition` と、接続されていない reporter block は event hat に含めません。「実行される内容」は hat を起点とする主要なカスタムブロック呼出し、broadcast、状態変更、表示操作を要約したもので、標準 block を含む全処理の逐語的な列挙ではありません。

## Stage

| target | ID | trigger            | 実行される内容                                                                                    |
|--------|----|--------------------|--------------------------------------------------------------------------------------------|
| Stage  | iM | green flag         | stop camera preview , stop pose recog , stop camera , hide all actors ;<br>showTitle 送信    |
| Stage  | i; | key space          | showCover 送信                                                                               |
| Stage  | i} | key down arrow     | finishTimedActorAction 送信、 skipMode=Down                                                   |
| Stage  | jb | key right arrow    | finishTimedActorAction 送信、 skipMode=Right                                                  |
| Stage  | jX | startStory 受信      | start camera , create sceneList , exec scene # %s with %s , create asset ,<br>create actor |
| Stage  | j/ | stopStory 受信       | stop camera , stop pose recog , show cover ; deleteAllActors , showMenu 送信                 |
| Stage  | j? | debugTestCamera 受信 | TMPose の camera preview を直接確認                                                              |
| Stage  | kD | showCover 受信       | show cover ; hidePrompt , deleteAllActors 送信                                               |
| Stage  | l= | Stage click        | 組み込み台本の有無に応じて showCover または startStory 送信                                                  |
| Stage  | l[ | showTitle 受信       | 実行 context を clear し、 hidePrompt , deleteAllActors 送信                                      |
| Stage  | m~ | stopKeyInput 受信    | Async Input の全 listener を停止                                                                |
| Stage  | nx | stopTouchInput 受信  | Async Input の全 listener を停止                                                                |

## Actor

| target | ID             | trigger                   | 実行される内容                                              |
|--------|----------------|---------------------------|------------------------------------------------------|
| Actor  | nY             | execActorAction 受信        | isTimeBasedAction , wait for actor action %s seconds |
| Actor  | oH             | deleteAllActors 受信        | clone を削除                                            |
| Actor  | oJ             | clone 開始                  | actorName 、位置、 scale を runtime envelope から初期化        |
| Actor  | actorFinishHat | finishTimedActorAction 受信 | 対象 actor と skipMode を照合して時間 action を完了               |



## UI sprite

| target              | ID                   | trigger                  | 実行される内容                            |
|---------------------|----------------------|--------------------------|------------------------------------|
| prompt              | oS                   | showPrompt 受信            | 案内 costume を表示                     |
| prompt              | oV                   | hidePrompt 受信            | 非表示                                |
| prompt              | oX                   | invalidScript 受信         | エラー costume を表示                    |
| openButton          | o!                   | green flag               | 非表示                                |
| openButton          | o%                   | sprite click             | file 選択後に hideMenu , startStory 送信 |
| openButton          | o*                   | hideMenu 受信              | 非表示                                |
| openButton          | o,                   | showMenu 受信              | ui.open skin で表示                   |
| reloadButton        | o.                   | green flag               | 非表示                                |
| reloadButton        | o:                   | hideMenu 受信              | 非表示                                |
| reloadButton        | o=                   | sprite click             | 保存済み台本で hideMenu , startStory 送信   |
| reloadButton        | o[                   | showMenu 受信              | 台本が保存済みなら表示                        |
| showTitleButton     | o`                   | green flag               | 非表示                                |
| showTitleButton     | ol                   | hideMenu 受信              | 非表示                                |
| showTitleButton     | o~                   | sprite click             | hideMenu , showTitle 送信            |
| showTitleButton     | pb                   | showMenu 受信              | title 以外なら表示                       |
| Loading             | pf                   | green flag               | 非表示                                |
| Loading             | pm                   | assetLoadingStarted 受信   | Loading costume を表示                |
| Loading             | pj                   | assetLoadingProgress 受信  | costume を循環                        |
| Loading             | ph                   | assetLoadingCompleted 受信 | 非表示、完了 sound                       |
| LoadingBubbleAnchor | loadingBubbleFlag    | green flag               | 非表示、bubble を clear                 |
| LoadingBubbleAnchor | loadingBubbleStarted | assetLoadingStarted 受信   | anchor を表示                         |

| target              | ID                     | trigger                  | 実行される内容                                     |
|---------------------|------------------------|--------------------------|---------------------------------------------|
| LoadingBubbleAnchor | loadingBubbleProgress  | assetLoadingProgress 受信  | runtime variable <code>message</code> を say |
| LoadingBubbleAnchor | loadingBubbleCompleted | assetLoadingCompleted 受信 | bubble を clear して非表示                        |

## 6.2 カスタムブロック定義一覧

引数名はprototypeの `argumentnames`、warpはprototypeの `mutation.warp` から取得します。「呼び出す処理／送信するmessage」には定義内で呼ぶ別のカスタムブロックや機能拡張の処理を示し、broadcast messageは送信する名前を明記します。

## 初期化・parse・共通処理

| target | ID | 定義                                             | 引数                          | warp | 呼び出す処理／送信する message                                                                                           |
|--------|----|------------------------------------------------|-----------------------------|------|---------------------------------------------------------------------------------------------------------------|
| Stage  | c: | init skinList<br>with %s                       | commaSeparatedText          | yes  | selectValue # %s separated by %s from %s                                                                      |
| Stage  | c_ | init poseList<br>with %s                       | commaSeparatedText          | yes  | selectValue # %s separated by %s from %s                                                                      |
| Stage  | dc | init soundList<br>with %s                      | commaSeparatedText          | yes  | selectValue # %s separated by %s from %s                                                                      |
| Stage  | gH | init<br>durationList<br>with %s                | commaSeparatedText          | no   | selectValue # %s separated by %s from %s                                                                      |
| Stage  | eM | selectValue #<br>%s separated by<br>%s from %s | index , separator ,<br>text | yes  | —                                                                                                             |
| Stage  | hI | substr of %s<br>after %s                       | text , firstDelim           | no   | —                                                                                                             |
| Stage  | dL | min %s %s                                      | valueA , valueB             | yes  | —                                                                                                             |
| Stage  | d) | exec command %s<br>%s                          | key , value                 | no   | substr of %s after %s , selectValue # %s<br>separated by %s from %s , setTMPoseURL with %s ;<br>invalidScript |
| Stage  | e+ | create<br>sceneList                            | —                           | yes  | selectValue # %s separated by %s from %s                                                                      |
| Stage  | fe | create asset                                   | —                           | no   | selectValue... ;<br>assetLoadingStarted/Progress/Completed                                                    |
| Stage  | fv | create actor                                   | —                           | no   | selectValue # %s separated by %s from %s                                                                      |

## camera・pose

| target | ID | 定義                      | 引数        | warp | 呼び出す処理／送信する message                                                                                                      |
|--------|----|-------------------------|-----------|------|--------------------------------------------------------------------------------------------------------------------------|
| Stage  | dQ | setTMPoseURL<br>with %s | URL       | no   | —                                                                                                                        |
| Stage  | dU | start camera            | —         | no   | —                                                                                                                        |
| Stage  | dX | stop camera             | —         | no   | —                                                                                                                        |
| Stage  | eD | start pose<br>recog     | —         | no   | —                                                                                                                        |
| Stage  | dG | stop pose recog         | —         | no   | —                                                                                                                        |
| Stage  | dY | rate of pose<br>recog   | —         | no   | —                                                                                                                        |
| Stage  | d! | label of pose<br>recog  | —         | no   | —                                                                                                                        |
| Stage  | d% | start camera<br>preview | —         | no   | —                                                                                                                        |
| Stage  | dC | stop camera<br>preview  | —         | no   | —                                                                                                                        |
| Stage  | dk | exec pose<br>action %s  | actorName | no   | start camera preview, start pose recog, exec pose %s,<br>stop pose recog, stop camera preview; showPrompt,<br>hidePrompt |
| Stage  | f[ | exec pose %s            | actorName | no   | change skin..., min..., rate of pose recog                                                                               |

## scene・action・actor

| target | ID           | 定義                                  | 引数                        | warp | 呼び出す処理／送信する message                                                                                                                                |
|--------|--------------|-------------------------------------|---------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Stage  | fB           | exec scene # %s<br>with %s          | sceneIndex ,<br>sceneData | no   | selectValue # %s separated by %s from<br>%s , substr of %s after %s , exec command<br>%s %s , exec actionList , hide all<br>actors ; invalidScript |
| Stage  | e=           | exec actionList                     | —                         | no   | exec action %s                                                                                                                                     |
| Stage  | fC           | exec action %s                      | action                    | no   | exec stage action %s , exec actor action<br>%s                                                                                                     |
| Stage  | gP           | exec stage action<br>%s             | action                    | no   | touchInputToChangeScene %s %s , exec<br>keyInputToChangeScene %s %s , exec<br>branch action %s , exec transition<br>action %s , wait %s seconds    |
| Stage  | gp           | exec actor action<br>%s             | action                    | no   | selectValue # %s separated by %s from<br>%s , 3つのlist初期化、 exec pose action<br>%s ; execActorAction                                                 |
| Stage  | ee           | hide all actors                     | —                         | no   | execActorAction                                                                                                                                    |
| Stage  | eh           | change skin of %s<br>to %s          | actorName ,<br>skinName   | no   | execActorAction                                                                                                                                    |
| Stage  | eR           | show cover                          | —                         | no   | selectValue # %s separated by %s from<br>%s ; showMenu                                                                                             |
| Actor  | it           | isTimeBasedAction                   | —                         | no   | —                                                                                                                                                  |
| Actor  | actorWaitDef | wait for actor<br>action %s seconds | seconds                   | no   | —                                                                                                                                                  |

## transition・branch・input

| target | ID | 定義                               | 引数                            | warp | 呼び出す処理／送信する message                                                    |
|--------|----|----------------------------------|-------------------------------|------|------------------------------------------------------------------------|
| Stage  | ga | exec transition action %s        | transitionName                | no   | exec transition reset, exec transition fadeUp, exec transition fadeOut |
| Stage  | gf | exec transition fadeOut          | —                             | no   | —                                                                      |
| Stage  | gh | exec transition fadeUp           | —                             | no   | —                                                                      |
| Stage  | gj | exec transition reset            | —                             | no   | —                                                                      |
| Stage  | g] | exec branch action %s            | branchName                    | no   | selectValue...                                                         |
| Stage  | hq | exec keyInputToChangeScene %s %s | keyIdList, sceneLabellist     | no   | Async Input                                                            |
| Stage  | hu | touchInputToChangeScene %s %s    | actorNameList, sceneLabellist | no   | Async Input                                                            |
| Stage  | hy | wait %s seconds                  | seconds                       | no   | More Timers                                                            |

## 6.3 主要な呼出し経路

| 起点           | 経路                                                                  |
|--------------|---------------------------------------------------------------------|
| green flag   | 初期化 → camera/pose停止 → actor非表示 → showTitle                          |
| startStory   | 台本検証 → create sceneList → asset/actor生成 → exec scene # %s with %s   |
| scene実行      | exec command %s %s → exec actionList → actionごとに exec action %s     |
| Stage action | branch、transition、key/touch入力、wait などへdispatch                      |
| Actor action | runtime envelope設定 → execActorAction → 対象clone → 移動・見た目・音・時間 action |
| pose action  | camera preview/pose認識開始 → exec pose %s 反復 → 認識停止 → prompt非表示        |
| 終了           | stopStory → camera/pose停止 → actor削除 → cover → menu                  |

## 7. broadcast と状態遷移

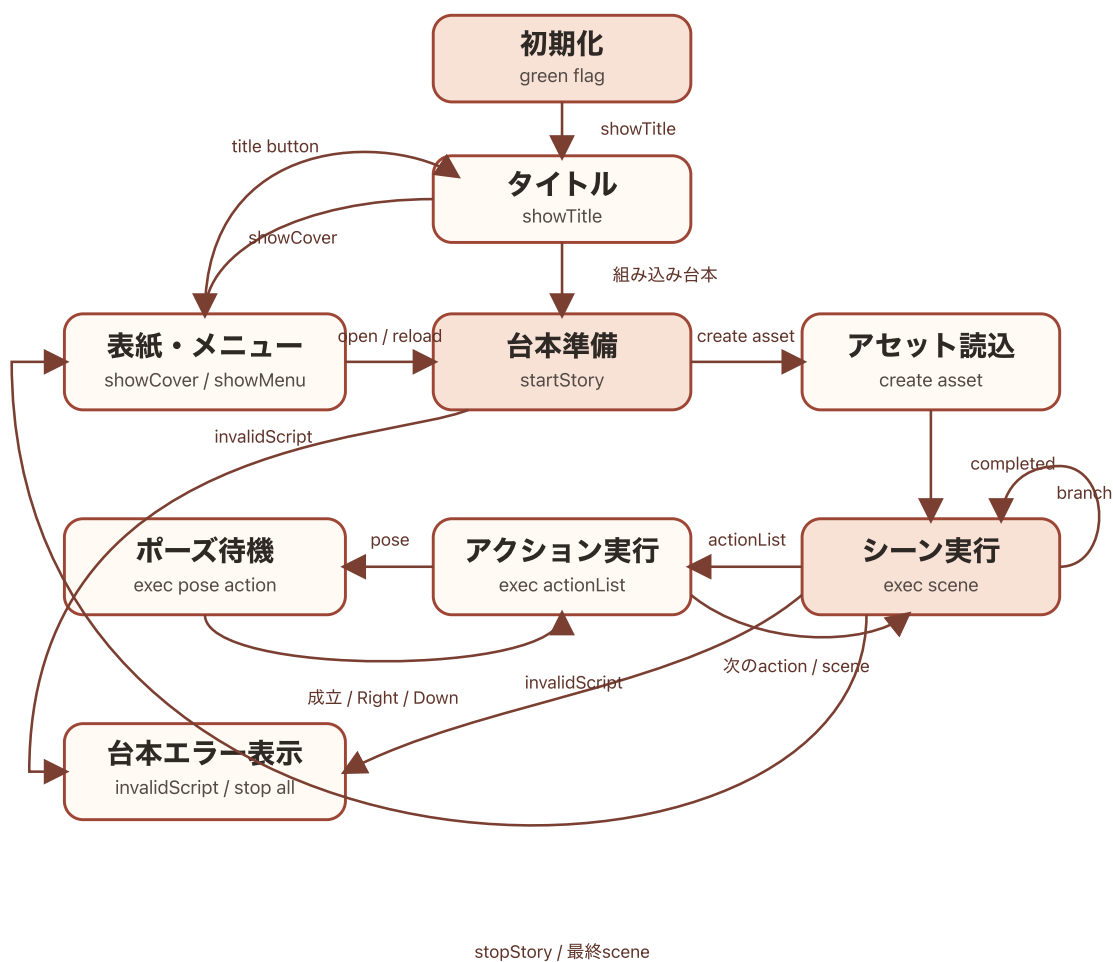
### 7.1 message 一覧

| message                             | 主な送信者                                                                       | 受信者                                                        | 役割                          |
|-------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------|
| <code>showPrompt</code>             | <code>Stage</code>                                                          | <code>prompt</code>                                        | 操作・pose 案内を表示               |
| <code>hidePrompt</code>             | <code>Stage</code>                                                          | <code>prompt</code>                                        | 案内を非表示                      |
| <code>invalidScript</code>          | <code>Stage</code>                                                          | <code>prompt</code>                                        | 台本エラーを表示                    |
| <code>hideMenu</code>               | 3つの menu button                                                             | 3つの menu button                                            | menu を一括非表示                 |
| <code>showMenu</code>               | <code>Stage</code>                                                          | 3つの menu button                                            | 利用可能な menu を表示              |
| <code>startStory</code>             | <code>Stage</code> , <code>openButton</code> ,<br><code>reloadButton</code> | <code>Stage</code>                                         | 台本の解析・実行を開始                 |
| <code>stopStory</code>              | <code>Stage</code>                                                          | <code>Stage</code>                                         | 実行を停止し cover へ戻す            |
| <code>showCover</code>              | <code>Stage</code>                                                          | <code>Stage</code>                                         | cover を構築して menu を表示        |
| <code>showTitle</code>              | <code>Stage</code> , <code>showTitleButton</code>                           | <code>Stage</code>                                         | title 状態へ戻す                 |
| <code>execActorAction</code>        | <code>Stage</code>                                                          | <code>Actor</code>                                         | action envelope を clone へ通知 |
| <code>deleteAllActors</code>        | <code>Stage</code>                                                          | <code>Actor</code>                                         | 全 clone を削除                 |
| <code>assetLoadingStarted</code>    | <code>Stage</code> / Asset Manager                                          | <code>Loading</code> ,<br><code>LoadingBubbleAnchor</code> | Loading 表示を開始               |
| <code>assetLoadingProgress</code>   | <code>Stage</code> / Asset Manager                                          | <code>Loading</code> ,<br><code>LoadingBubbleAnchor</code> | 進捗 costume と message を更新    |
| <code>assetLoadingCompleted</code>  | <code>Stage</code> / Asset Manager                                          | <code>Loading</code> ,<br><code>LoadingBubbleAnchor</code> | Loading 表示を終了               |
| <code>stopKeyInput</code>           | Async Input                                                                 | <code>Stage</code>                                         | key listener を停止            |
| <code>stopTouchInput</code>         | Async Input                                                                 | <code>Stage</code>                                         | touch listener を停止          |
| <code>finishTimedActorAction</code> | <code>Stage</code> の Right/Down key<br>hat                                  | <code>Actor</code>                                         | 時間 action を確定状態へ進める         |

| message         | 主な送信者                    | 受信者   | 役割                 |
|-----------------|--------------------------|-------|--------------------|
| debugTestCamera | TurboWarp editor からの手動送信 | Stage | camera preview の診断 |

stopKeyInput と stopTouchInput は標準 broadcast block ではなく、Async Input へ渡した callback message です。 debugTestCamera は通常フローに送信元を持たない診断用 message です。

## 7.2 状態遷移



### 紙芝居アプリの主要状態遷移

主要状態はStageが所有します。UI表示そのものを状態の正本にせず、runtime variable、 broadcast、実行中の custom block から導出します。



| 状態            | 入口                                     | 主な出口                                                    |
|---------------|----------------------------------------|---------------------------------------------------------|
| 初期化           | green flag                             | showTitle                                               |
| title         | showTitle                              | 組み込み台本なら Stage click で startStory、それ以外は cover           |
| cover/menu    | showCover または stopStory                | open/reload で startStory、title button で showTitle       |
| 台本準備          | startStory                             | 正常なら asset loading と scene 実行、検証異常なら invalidScript      |
| asset loading | create asset                           | assetLoadingCompleted 後に scene 実行                       |
| scene 実行      | exec scene # %s with %s                | 次 scene/branch、最終 scene で stopStory、解析異常で invalidScript |
| action 実行     | exec actionList                        | Right で action 境界、Down で scene 境界、完了で次 action           |
| pose 待機       | exec pose action %s                    | pose 成立、Right/Down で action 実行へ戻る                       |
| 台本エラー表示・実行停止  | 台本・command・scene 解析エラー時の invalidScript | prompt が ui.invalidScript を表示し、送信元の stop all で実行を停止     |

invalidScript は pose 待機への遷移ではありません。Stage は台本検証、command 解析、scene 解析のエラー時にこの message を送信し、各送信箇所の直後に stop all を実行します。prompt は message を受信すると ui.invalidScript の skin を設定して表示します。

skipMode は要求、skipContext は消費可能な境界です。要求は title、action、pose、scene の該当境界だけが消費し、scene 開始、cover、stop で clear します。nextSceneLabel は key/touch listener が設定し、scene loop が label を index へ解決したあと削除します。

## 8. 検証

### 8.1 変更対象ごとのテスト

| 変更対象            | 主なテスト                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|
| builder API/CLI | <code>test/builder.test.mjs</code>                                                                                                 |
| 展開SB3の構造        | <code>test/sb3-project.test.mjs</code> 、 <code>test/skip-mode.test.mjs</code>                                                      |
| 内部仕様書の構造一覧      | <code>test/internal-specification.test.mjs</code>                                                                                  |
| TurboWarp 実行結果  | <code>test/turbowarp-vm.test.mjs</code>                                                                                            |
| 入力、分岐、wait      | <code>test/async-input.test.mjs</code> 、 <code>test/register-branch.test.mjs</code> 、 <code>test/wait-action.test.mjs</code>       |
| 文書、画像、ライセンス     | <code>test/docs-config.test.mjs</code> 、 <code>test/docs-images.test.mjs</code> 、 <code>test/documentation-license.test.mjs</code> |
| 公開物と汎用性         | <code>test/sb3-publication.test.mjs</code> 、 <code>test/build-freshness.test.mjs</code>                                            |

### 8.2 標準チェック

```
pnpm lint
pnpm format
pnpm typecheck
pnpm test
pnpm run build
```

GitHub Actions は clean な Linux 環境で `pnpm install --frozen-lockfile`、`pnpm test`、`pnpm build` を実行します。ローカルで成功しても、未追跡ファイルや既存生成物へ依存していないことを CI で確認します。

`pnpm run build` は少なくとも次を生成・検証します。

- `dist/downloads/kamishibai.sb3`
- 一般文書ごとの Web Publication、HTML/PDF、Vivliostyle CLI が `h2`・`h3` までから生成する目次
- 参加者向け・スタッフ向け体験会資料
- 公開サイトのリンク、画像、目次、PDF bookmark、favicon

SB3 または runtime を変更した場合は、生成 SB3 を TurboWarp で開いて次を手動確認します。

- 読みエラーがない
- green flag で title と menu が表示される

- 外部台本と組み込み台本の対象フローが開始できる
- pause、Space、Right、Downの進行が意図どおり動く
- Loading、画像、音声、テキストが正しく表示・再生される

内部構造を変更したPRでは、`app/project.source.json` と本書の target、変数、message、hat、custom block 一覧を同時に更新します。

## 9. 公開

### 9.1 GitHub Pages

```
pnpm run deploy
```

`predeploy` がフル build を行い、成功した `dist/` だけを `gh-pages` へ公開します。公開後は top page、文書一覧、各カードの HTML/Vivliostyle Viewer/PDF、SB3 download を実際の URL から確認します。

問題がある場合は、直前の検証済み commit を checkout した clean な環境から再度 build・deploy します。生成済み `dist/` だけを手作業で修正しません。

### 9.2 npm パッケージ

公開済み version は変更・再利用できません。release ごとに新しい version と Git tag を使います。

1. `package.json`、lockfile、`src/builder/constants.js`、README の導入例を同じ version へ更新する。
2. clean な commit で標準チェック、フル build、公開内容の dry-run を実行する。

```
pnpm release:check
```

3. tarball のファイル一覧、license、size、CLI/API を確認する。
4. Git worktree ではなく通常の clean clone から公開する。
5. WebAuthn などの認証を完了して public package として公開する。

```
npm publish --access public
```

6. registry 反映後に metadata を確認する。

```
npm view @kubohiroya/tmpose-kamishibai@<VERSION> \
  version license dist-tags.latest dist.integrity --json
```

7. 一時ディレクトリへ公開版を導入し、CLIの `--version` と `@kubohiroya/tmpose-kamishibai/builder` の `import`を確認する。
8. 公開に使った確定 commitへ annotated tag を作り、GitHub Release を作成する。

公開後に問題が見つかった場合は対象 version を `npm deprecate` し、修正版を新しい patch version として公開します。公開済み tarball や tag を差し替えません。

## 10. トラブルシューティング

| 症状                                                                                       | 確認と対応                                                                                                          |
|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>sb3:import</code> が置換を拒否する                                                         | <code>git status</code> と <code>git diff -- app</code> を確認し、 <u>toolchainの手順</u> に従う                           |
| <code>.app.rollback-*</code> や <code>.&lt;出力名</code><br><code>&gt;.rollback-*</code> が残る | 削除前に元出力と比較し、 <u>toolchainの失敗時の扱い</u> に従う                                                                       |
| 埋め込み拡張が追跡refと異なる                                                                         | <code>pnpm sb3:extensions:status</code> で確認し、固定 commit へ戻すなら <code>sync</code> 、更新するなら <code>update</code> を使う |
| PDF 生成 browser が見つからない                                                                   | Chrome/Chromium を導入し、必要なら <code>VIVLIOSTYLE_CHROME_PATH</code> を設定する                                           |
| ローカルだけ test が通る                                                                          | 生成物と未追跡ファイルを確認し、clean clone と <code>pnpm install --frozen-lockfile</code> で再現する                                |
| builder が既存出力を更新しない                                                                      | エラーの <code>stage</code> 、asset 名、URI を確認する。rollback 領域が残っていないか確認する                                             |
| 公開直後に npm registry が 404 になる                                                             | 同じ version を再publishせず、npm 公開 page と registry の反映を待つ確認する                                                       |

復旧で Git 履歴を破壊しません。公開済み変更は `git revert` または新しい修正 PR で戻し、tag を 移動しません。

## 11. ライセンスと秘密情報

| 対象                                   | ライセンス                                              |
|--------------------------------------|----------------------------------------------------|
| <code>docs/general/**</code>         | CC BY-SA 4.0                                       |
| <code>docs/workshops/**</code>       | Copyright © 2026 Hiroya Kubo. All rights reserved. |
| 上記以外で個別表示のない、本プロジェクトが著作権を持つソフトウェアと素材 | MPL-2.0                                            |

詳細は [LICENSES.md](#)、[docs/general/LICENSE.md](#)、[docs/workshops/LICENSE.md](#) を参照してください。

第三者の画像、音声、font、model、機能拡張には個別の license または利用条件が適用されます。builder で組み込む素材は asset manifest の `license` へ由来を記録します。許諾が確認できない 素材を本体または sample へ追加しません。

token、npm 認証情報、秘密鍵、個人情報を repository、SB3、台本、manifest、生成 HTML へ 記録しません。認証情報は環境変数、OS の keychain、GitHub Secrets など、公開物へ含まれない 仕組みで渡します。

## 12. 関連ドキュメント

- [03-user-guide.md](#): アプリの利用方法と成果物の使い分け
- [04-dsl-manual.md](#): 台本の構造と書き方
- [05-command-reference.md](#): コマンドと action の外部仕様
- [06-developer-guide.md](#): setup と変更手順
- [history.md](#): DSL とアプリの変更履歴
- [README.md](#): プロジェクト全体の入口